

WORKFORCEOS

Enterprise Time, Attendance & Workforce Management Platform

Complete Technical Product Requirements Document

Frontend + Backend + Business Process

Document purpose	Portfolio-grade blueprint designed to demonstrate senior Java/backend engineering for an enterprise workforce-management employer.
Implementation status	Specification only - no production code is included.
Backend baseline	Java 25 LTS, Spring Boot 4.1.x, Spring Modulith, PostgreSQL 18.
Frontend baseline	React 19.2.x, TypeScript 6, Vite 8, Material UI 9.
Architecture	Modular monolith with explicit domain boundaries and extraction path to services.
Document date	14 August 2026

Designed to showcase domain modeling, SOLID architecture, multi-tenancy, concurrency, time-zone correctness, auditability, enterprise security, testing, and production readiness.

Document Control

Item	Definition
Product name	WorkforceOS
Document type	Technical Product Requirements Document (Technical PRD)
Primary audience	Hiring team, engineering leadership, backend/frontend engineers, reviewers
Primary objective	Demonstrate credible senior-engineering decisions for a multinational time & attendance platform
Delivery scope	Business process, requirements, architecture, frontend, backend, data, security, testing, operations, roadmap
Explicit exclusion	This PRD does not implement application source code.

Table of Contents

1. Executive Summary
2. Product Vision, Goals and Scope
3. Personas and Access Model
4. End-to-End Business Processes
5. Functional Requirements
6. Domain Rules and Edge Cases
7. Backend Technical Architecture
8. SOLID and Codebase Design Rules
9. Data Architecture
10. API and Integration Design
11. Frontend Technical Architecture
12. UX / UI Requirements
13. Security and Compliance Controls
14. Non-Functional Requirements
15. Testing and Quality Strategy
16. Observability and Operations
17. DevSecOps and Deployment
18. Implementation Roadmap
19. Demo and Interview Narrative
20. Acceptance Criteria
21. Risks and Trade-offs
22. Reference Technology Baseline

1. Executive Summary

WorkforceOS is a multi-tenant, time-zone-aware enterprise workforce management platform focused on the hardest engineering problems in time and attendance: scheduling, raw clock-event ingestion, attendance calculation, configurable policies, exceptions, approvals, payroll readiness, auditability, and multinational operation.

The product is intentionally designed as a portfolio project that could plausibly serve enterprise customers. **Its purpose is not to recreate an entire HR suite.** Instead, it demonstrates the architectural judgment and engineering depth expected from a senior Java engineer working on a mature workforce platform.

Portfolio thesis A recruiter can see the UI in minutes, but an engineering interviewer should be able to spend an hour discussing domain invariants, time-zone edge cases, data isolation, idempotency, concurrency, testing, and deployment decisions.

1.1 Key Differentiators

- Multi-country and multi-time-zone behavior is a first-class domain concern, not a formatting afterthought.
- Raw time events are immutable; derived attendance records can be recalculated with full traceability.
- Attendance rules are extensible through small strategy interfaces rather than large conditional services.
- Tenant isolation is enforced throughout authentication, authorization, persistence, caching, and audit paths.
- Cross-midnight shifts, daylight-saving transitions, retries, duplicate punches, late events, and concurrent approvals are explicitly specified.
- Frontend architecture separates server state, form state, URL state, and local UI state instead of using a single global store.
- Automated tests focus on domain behavior and module boundaries, not only controller coverage.

1.2 Recommended Technology Baseline

Layer	Decision	Rationale
Runtime	Java 25 LTS	Modern LTS baseline; demonstrates current Java capability without adopting a short-lived feature release.
Backend	Spring Boot 4.1.x	Current generation compatible with Java 25; strong ecosystem for enterprise APIs.
Architecture	Spring Modulith + modular monolith	Explicit domain modules, internal events, module verification, evolutionary path to services.
Persistence	PostgreSQL 18	Strong relational integrity, indexing, partitioning options, JSON support, mature operations.
Cache / ephemeral	Redis	Rate limiting, short-lived caches, distributed coordination only where justified.
Frontend	React 19.2 + TypeScript 6 + Vite 8	Component-based enterprise SPA with strict typing and fast tooling.

Layer	Decision	Rationale
UI system	Material UI 9	Accessible, consistent enterprise component baseline.
Server state	TanStack Query	Request lifecycle, caching, invalidation, retry, and async server-state ownership.
Identity	OIDC/OAuth 2.0; Spring Security resource server	Enterprise SSO ready; bearer-token API protection.
Observability	OpenTelemetry + metrics/logging backend	Traces, metrics, structured logs, correlation IDs.
Packaging	Docker / OCI images	Reproducible runtime and cloud portability.

2. Product Vision, Goals and Scope

2.1 Product Vision

Vision: Give multinational employers a trustworthy record of who was expected to work, what time was actually captured, how policies converted those events into payable attendance, which exceptions require human decision, and exactly who changed or approved what.

2.2 Business Goals

ID	Goal	Success signal
BG-01	Correctly calculate attendance across configured time zones and shift patterns	Deterministic test suite passes all time-zone/cross-midnight/DST scenarios.
BG-02	Reduce manager effort spent finding attendance anomalies	Exception queue surfaces only records that require intervention.
BG-03	Provide payroll-ready approved time data	Closed period exports contain only finalized records and can be reproduced.
BG-04	Support multiple enterprise tenants securely	No cross-tenant access under automated authorization and persistence tests.
BG-05	Demonstrate senior engineering quality	Architecture, tests, docs, telemetry, CI/CD and trade-offs are reviewable in the repository.

2.3 In Scope - MVP

- Tenant, user, role, organization, location and employee administration.
- Employment assignment and manager hierarchy.
- Shift templates and employee schedules.
- Manual/web clock-in and clock-out; import/API ingestion model for external terminals.
- Attendance calculation and configurable grace/overtime/break policies.

- Exceptions: late, early leave, absent, missing punch, overtime, unscheduled work, holiday/rest-day work.
- Correction requests and manager review.
- Attendance approval workflow and payroll-period close.
- CSV payroll export with idempotent export history.
- Immutable audit trail and operational dashboards.
- Notifications for actionable exceptions and approval states.

2.4 Explicitly Out of Scope for MVP

- Full payroll tax calculation, salary disbursement, benefits, recruitment, performance management.
- Biometric face/fingerprint implementation; only device/integration interfaces are specified.
- Native mobile applications; responsive web is sufficient for MVP.
- AI-based anomaly scoring.
- Automatic labor-law compliance engine for every country. Policies are configurable, but legal correctness remains tenant responsibility.
- Microservice decomposition before operational evidence justifies it.

3. Personas and Access Model

Persona	Primary tasks	Access boundary
Employee	View schedule; clock in/out; view own attendance; submit correction	Own records only, plus self-service actions.
Manager	View team attendance; review exceptions; approve/reject corrections and overtime	Direct/indirect reports according to configured org scope.
HR Admin	Maintain people, schedules, calendars, policies; review company-wide exceptions	Tenant-wide workforce data except restricted payroll actions.
Payroll Admin	Review approved totals; close periods; export payroll data	Approved/closed attendance and payroll artifacts.
Tenant Admin	Manage tenant configuration, SSO, roles, integrations	Tenant configuration and delegated administrative access.
System Operator	Operate platform, inspect health, troubleshoot by support workflow	No default business-data browsing; privileged support is audited and time-limited.

3.1 Authorization Principles

- Deny by default.
- Every request has an authenticated subject and tenant context.
- Role grants capability; organizational scope narrows which business objects are visible.
- Sensitive actions require explicit permissions, not UI hiding.
- Payroll period close, policy publication, role changes, and support impersonation generate security-grade audit events.

4. End-to-End Business Processes

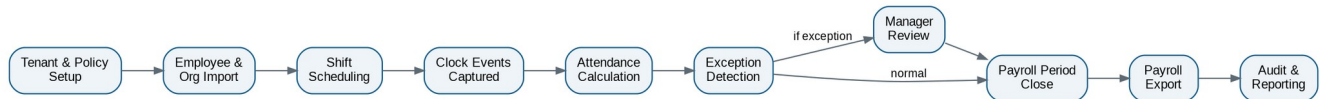


Figure 1. End-to-end business flow.

4.1 Tenant Onboarding and Configuration

- STEP** Create tenant with canonical identifier, display name, default locale, primary time zone, data-retention policy, and feature flags.
- STEP** Configure identity provider / SSO parameters or local development identity for demo mode.
- STEP** Create organization units, legal entities, work locations, holiday calendars and pay-period cadence.
- STEP** Define attendance policy versions. A published policy is immutable; future changes create a new effective-dated version.
- STEP** Import employees and employment assignments with manager relationships and assigned policy/calendar.
- STEP** Validate configuration completeness before allowing payroll period close.

4.2 Scheduling Process

- STEP** HR/manager chooses a shift template or creates a one-off schedule entry.
- STEP** System validates overlap, employment status, location/time-zone context, and scheduling horizon.
- STEP** Schedule is stored using local business semantics plus IANA ZoneId; derived instants are calculated for execution.
- STEP** Schedule changes after clock events exist remain versioned/audited and trigger attendance recalculation when relevant.

4.3 Clock Event Capture

- STEP** Client obtains current employee/tenant context and permitted clock action.
- STEP** Client submits event with idempotency key, client timestamp, server-received timestamp, source, device metadata and optional location metadata.
- STEP** Backend validates authorization, acceptable event window, event uniqueness and tenant ownership.
- STEP** Raw event is appended; it is not overwritten by later corrections.
- STEP** Attendance recalculation is triggered for the impacted employee/business date.

4.4 Attendance Calculation and Exception Workflow

- STEP** Resolver finds the applicable shift/schedule, approved leave, holiday calendar and effective attendance policy.
- STEP** Calculator pairs relevant events and derives worked intervals, break duration, payable/regular/overtime minutes and attendance status.
- STEP** Rules evaluate the calculated facts and emit zero or more exceptions.
- STEP** Normal records can auto-approve if tenant policy allows. Exception records enter manager review.
- STEP** Manager may approve, reject or request correction. Every decision records actor, reason, timestamp and record version.

STEP A correction adds an adjustment or corrected event; source events remain immutable and attendance is recalculated.

4.5 Payroll Period Close

STEP Payroll Admin opens a pay period and reviews readiness summary.

STEP System blocks close if required records are unresolved, unless an authorized override policy explicitly permits it.

STEP Close action snapshots/references finalized attendance totals and locks normal edits for the period.

STEP Export is generated with deterministic content and export checksum/version.

STEP Re-export returns the same logical dataset unless a formal reopen/amendment workflow occurs.

4.6 Reopen / Amendment Process

Control objective Closed-period data must never be silently mutated. A reopen requires elevated permission, mandatory reason, audit event, and a new export version after re-approval.

5. Functional Requirements

ID	Module	Requirement	Priority
FR-TEN-01	Tenant Management	Create/update tenant configuration; tenant ID immutable after creation.	P0
FR-IAM-01	Authentication	Authenticate through OIDC/OAuth 2.0; development profile may use local test identity.	P0
FR-IAM-02	Authorization	RBAC plus organizational scope and tenant boundary.	P0
FR-PEO-01	Employee	CRUD employee profile with effective-dated employment assignment.	P0
FR-ORG-01	Organization	Maintain legal entities, org units, locations and reporting hierarchy.	P0
FR-SCH-01	Scheduling	Create shift templates and employee schedule entries; detect overlap.	P0
FR-TIM-01	Clock Event	Capture clock in/out/break events with idempotency.	P0
FR-TIM-02	Clock Import	Ingest batch/API events from external source with source-event identity.	P1
FR-ATT-01	Attendance	Calculate worked, break, regular	P0

ID	Module	Requirement	Priority
		and overtime minutes.	
FR-POL-01	Policy	Maintain effective-dated attendance policy versions and rule parameters.	P0
FR-EXC-01	Exceptions	Detect, list, filter and prioritize attendance exceptions.	P0
FR-APR-01	Approval	Approve/reject exception/correction with optimistic locking.	P0
FR-LEV-01	Leave	Record approved leave windows affecting expected attendance.	P1
FR-HOL-01	Holiday	Assign effective holiday calendar by employee/location/legal entity.	P0
FR-PAY-01	Pay Period	Open, validate, close and reopen with audited authority.	P0
FR-PAY-02	Export	Generate payroll CSV and export history/checksum.	P0
FR-AUD-01	Audit	Search immutable audit events by actor/entity/action/time.	P0
FR-REP-01	Dashboard	Operational counts for attendance, exceptions, approvals and overtime.	P1
FR-NOT-01	Notification	Notify managers/employees for configured actionable state changes.	P1

5.1 Dashboard Requirements

- Today: scheduled, present, absent, late, on leave, open exceptions, overtime minutes.
- Manager queue: exceptions requiring action, ordered by SLA/severity and age.
- Payroll readiness: percentage finalized, unresolved blockers, total regular/overtime/holiday hours.
- All metrics must be tenant- and permission-scoped and link to the filtered underlying records.

6. Domain Rules and Edge Cases

Rule ID	Scenario	Required behavior
DR-01	Cross-midnight shift	Business date is anchored to scheduled shift start (configurable policy); event matching spans the shift window.
DR-02	DST spring forward	Elapsed duration uses instants; local wall-clock discontinuity must not create phantom

Rule ID	Scenario	Required behavior
		worked time.
DR-03	DST fall back	Repeated local time remains unambiguous because source is normalized to Instant plus ZoneId.
DR-04	Duplicate submission	Same idempotency key/source event ID returns the original logical result; no duplicate raw event.
DR-05	Out-of-order event	Late-arriving raw event may trigger deterministic recalculation of impacted date/shift.
DR-06	Missing clock-out	Attendance is incomplete and creates exception; system never invents a final punch silently.
DR-07	Schedule changed later	Recalculate using effective schedule version and preserve prior audit/revision evidence.
DR-08	Policy change	Published historical policy version remains attached to historical calculation unless explicit reprocessing is authorized.
DR-09	Concurrent approval	Only one update wins expected record version; stale reviewer receives conflict and current state.
DR-10	Closed payroll period	Normal mutations blocked; reopen/amendment flow required.
DR-11	Tenant boundary	Identifiers from another tenant behave as inaccessible; no information leakage through error details.
DR-12	Unscheduled work	Create attendance record and exception; manager decides whether payable/approved according to policy.

6.1 Time Representation Rules

- Persist event instants in UTC using a type semantically equivalent to Java Instant.
- Persist IANA time-zone identifiers (for example Asia/Jakarta or Europe/Berlin), not numeric offsets as the long-term source of truth.
- Persist business date separately where reporting/payroll semantics require it.
- Never derive payable duration by subtracting unqualified local date-time values across time-zone transitions.
- API responses include ISO-8601 instants and explicit business-time-zone metadata.

6.2 Policy Rule Categories

Category	Examples	Output
Arrival	Grace minutes, early-clock window, late threshold	late_minutes, LATE exception
Departure	Early leave tolerance, minimum worked time	early_leave_minutes, exception
Break	Minimum break after N hours, unpaid break rules	paid/unpaid break minutes, violation
Overtime	Daily/weekly threshold, approval requirement	overtime buckets and approval state
Holiday / Rest Day	Multiplier metadata or payable category	holiday/rest-day minutes
Absence	No qualifying time events and no approved leave	ABSENT exception
Rounding	Optional tenant-configured rounding for payroll output	rounded payable minutes; raw facts retained

7. Backend Technical Architecture

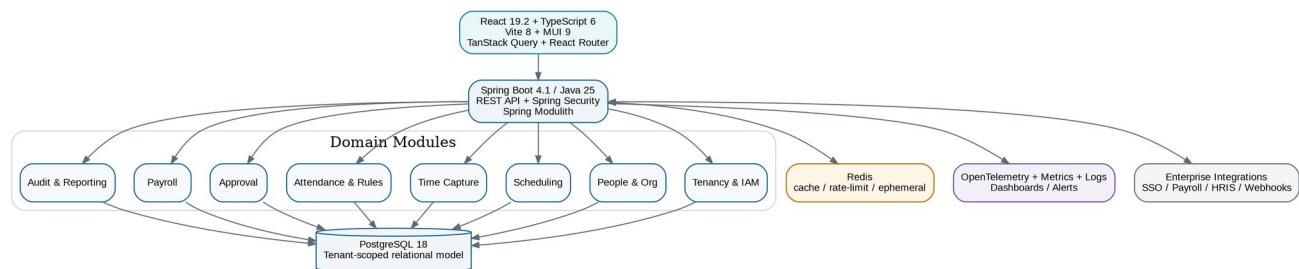


Figure 2. Target logical architecture.

7.1 Architectural Style: Modular Monolith

Decision: Deploy one Spring Boot application for the first production-grade version while enforcing independently testable domain modules. The deployment boundary is intentionally larger than the domain-module boundaries.

7.2 Why Not Microservices First?

- Time and attendance transactions are strongly relational and cross several domain concepts; premature distribution would add network failure modes before independent scaling needs exist.
- One deployable simplifies local development, integration tests, transactions and demo operations.
- Module boundaries, application events, ports and database ownership conventions preserve an extraction path.
- A future service extraction requires evidence: independent scaling pressure, release ownership, fault isolation, regulatory segregation or materially different availability requirements.

7.3 Domain Modules

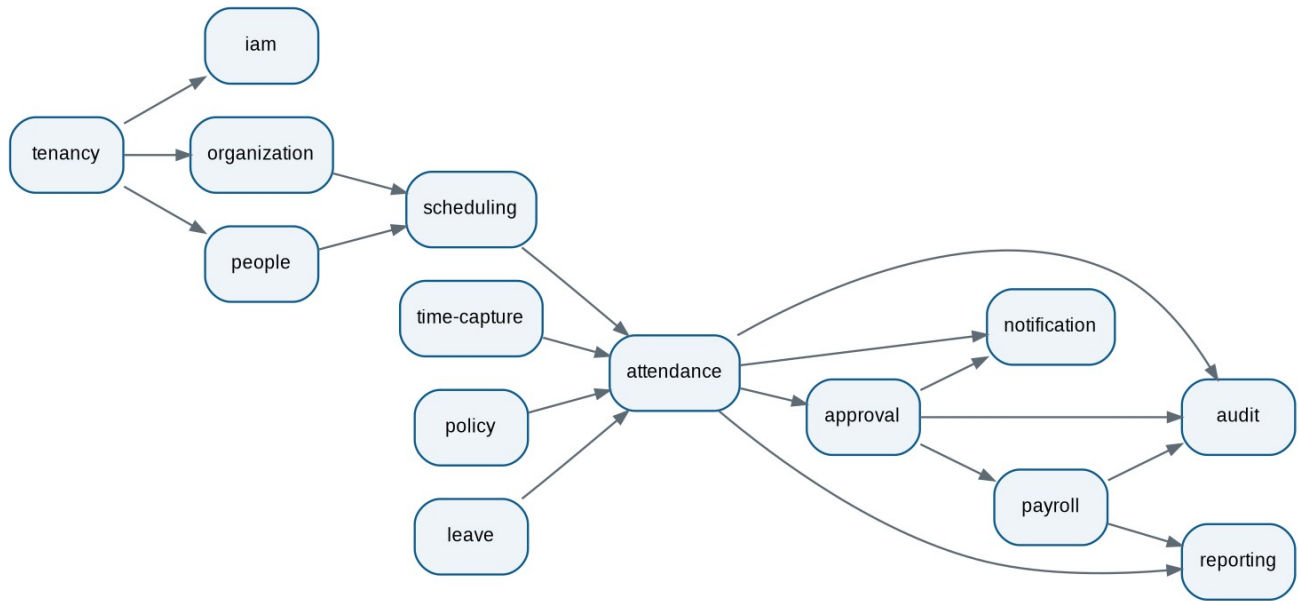


Figure 3. Proposed module dependency direction. Cycles are prohibited.

Module	Responsibilities	Owns
tenancy	Tenant lifecycle, configuration, feature availability	tenant, tenant_setting
iam	Identity mapping, roles, permissions, org scope	user_account, role_binding
people	Employee and employment assignment	employee, employment_assignment
organization	Legal entity, org unit, work location	legal_entity, org_unit, work_location
scheduling	Shift templates and scheduled work	shift_template, schedule_entry
time-capture	Immutable raw events, sources, idempotency	time_event, ingestion_request
policy	Policy versions and typed parameters	attendance_policy, policy_version
attendance	Calculation, intervals, totals, exceptions	attendance_record, work_interval, attendance_exception
leave	Approved absence windows	leave_request / leave_window
approval	Workflow decisions and correction requests	approval_case, approval_action
payroll	Pay periods, close/reopen, exports	pay_period, payroll_export
audit	Immutable actor/action/entity audit stream	audit_event
notification	Notification intents and delivery state	notification
reporting	Read-optimized projections / queries	report projections / views

7.4 Layering Inside Each Module

Layer	Allowed responsibility	Dependency direction
Domain	Entities, value objects, domain services, invariants, domain events	Depends on Java/domain abstractions only.
Application	Use cases, transaction boundaries, orchestration, ports	Depends on domain and interfaces.
Adapter - inbound	REST controllers, message consumers, schedulers	Depends on application contracts.
Adapter - outbound	JPA repositories, identity provider, file export, notification client	Implements application/domain ports.
Configuration	Spring wiring, module configuration	Composes implementations; no business logic.

7.5 Transaction Strategy

- A single user use-case is transactional at the application-service boundary.
- Raw clock-event append and attendance recalculation may be one transaction for simple synchronous capture, or two steps with durable event publication for higher throughput.
- Cross-module interaction uses public application interfaces or domain/application events; modules may not reach into another module repository.
- External side effects (email/webhook/export delivery) occur after durable state commit and are retried idempotently.

8. SOLID and Codebase Design Rules

Mandatory engineering constraint Backend implementation must visibly demonstrate SOLID through module boundaries and small abstractions, not merely mention SOLID in README documentation.

Principle	WorkforceOS application	Review evidence
Single Responsibility	Separate schedule resolver, event matcher, attendance calculator, overtime policy, exception detector, approval policy and export formatter.	Classes have one reason to change; orchestration does not own calculations.
Open/Closed	Attendance rules implement stable rule contracts and are composed from configuration. New rule type does not require rewriting the calculation core.	Add a rule implementation + registration/configuration, not a new branch in a God service.
Liskov Substitution	Rule/port implementations honor common preconditions, result semantics and error contracts.	Contract tests run against multiple implementations.
Interface Segregation	Use small ports: EmployeeReader,	Consumers depend only on methods they

Principle	WorkforceOS application	Review evidence
	ScheduleReader, TimeEventAppender, AuditWriter, PayrollExporter instead of a giant WorkforceRepository.	need.
Dependency Inversion	Domain/application logic references ports and domain interfaces; frameworks and persistence remain adapters.	Core package has no dependency on controller/JPA implementation types.

8.1 Additional Design Guardrails

- No generic BaseService/BaseRepository hierarchy unless a concrete domain need is proven.
- No business logic in REST controllers or JPA entity callbacks.
- Prefer immutable value objects for TenantId, EmployeeId, TimeEventId, BusinessDate, Minutes and policy identifiers.
- Avoid primitive obsession for time, money-like payroll quantities, tenant IDs and status transitions.
- Use explicit result/status types for expected domain outcomes; exceptions are reserved for exceptional/technical conditions.
- Package visibility and module verification should prevent accidental coupling.
- No direct SQL/JPA relation traversals across module ownership boundaries without an explicitly designed read model.

8.2 Conceptual Rule Extension Model (No Implementation Code)

Abstraction	Purpose
AttendanceRule	Evaluates calculated attendance context and returns zero or more typed findings.
OvertimePolicy	Converts eligible worked minutes into regular/overtime buckets.
EventPairingPolicy	Determines valid work intervals from ordered immutable events.
ScheduleResolutionPolicy	Resolves the applicable schedule when planned work and actual events overlap.
PayrollExporter	Transforms closed-period payroll projection into external format.
NotificationChannel	Delivers an already-created notification intent to an external channel.

9. Data Architecture

9.1 Tenancy Model

Recommended MVP model: shared PostgreSQL database and shared schema with mandatory tenant_id on tenant-owned tables. Composite unique constraints and indexes include tenant_id. Authorization context must never rely on a client-supplied tenant ID without matching the authenticated claim.

9.2 Logical Data Model

Entity	Key fields / relationships	Notes
tenant	id, code, default_zone, locale, status	Root security and configuration boundary.
employee	id, tenant_id, employee_no, identity data, status	Employee number unique within tenant.
employment_assignment	employee_id, org_unit_id, manager_id, policy_id, effective range	Effective-dated assignment.
shift_template	tenant_id, local_start, local_end, break config, zone strategy	Reusable planned work pattern.
schedule_entry	employee_id, business_date, zone_id, planned start/end instants	Version/audit on changes.
time_event	employee_id, event_type, occurred_at, received_at, zone_id, source, source_event_id	Immutable append-only source facts.
attendance_record	employee_id, business_date, schedule ref, status, calculated totals, version	Derived and recalculable.
attendance_exception	record_id, type, severity, state, details	Actionable anomaly.
approval_case	subject type/id, state, current version	Optimistic locking target.
attendance_policy_version	policy_id, version, effective range, parameters	Published versions immutable.
pay_period	date range, state, closed_by, closed_at	Open -> validating -> closed -> reopened.
payroll_export	period_id, version, checksum, format, generated_by	Reproducible export history.
audit_event	tenant, actor, action, entity, before/after digest, correlation, timestamp	Append-only audit evidence.

9.3 Indexing Strategy

- attendance_record: (tenant_id, business_date, employee_id) and (tenant_id, status, business_date).
- time_event: (tenant_id, employee_id, occurred_at), unique (tenant_id, source, source_event_id) where applicable.
- attendance_exception: (tenant_id, state, severity, created_at).
- schedule_entry: (tenant_id, employee_id, business_date) plus overlap validation strategy.
- audit_event: (tenant_id, entity_type, entity_id, occurred_at) and (tenant_id, actor_id, occurred_at).

9.4 Data Integrity Rules

- Foreign keys are used within owned relational boundaries.
- Database unique constraints backstop idempotency and natural uniqueness.
- Optimistic version columns protect concurrent mutable aggregate updates.
- Soft delete is not a default. Business-effective records use status/effective dates; source/audit facts are retained according to retention policy.
- PII and security-sensitive fields are classified and excluded from ordinary application logs.

10. API and Integration Design

10.1 REST Conventions

- Base path: /api/v1.
- JSON request/response; ISO-8601 time formats; explicit zone identifiers when business-local context matters.
- Problem Details-style error body with stable machine code, safe message, correlation ID and field violations.
- Pagination uses stable sort and cursor where high-volume streams benefit; offset pagination acceptable for bounded admin tables.
- Idempotency-Key required for user/device operations that may be retried, especially clock events and exports.
- ETag/version or explicit version field for mutable approval/correction resources.

10.2 Representative API Surface

Method	Endpoint	Purpose
GET	/me	Current identity, tenant, permissions and scope.
GET	/employees	Search permission-scoped employees.
GET	/employees/{id}/schedule	View schedule range.
POST	/time-events	Append a clock/break event idempotently.
GET	/attendance	Filter attendance records.
GET	/attendance/{id}	Attendance detail, intervals, exceptions, audit summary.
POST	/attendance/{id}/corrections	Submit correction request.
POST	/approval-cases/{id}/approve	Approve using expected version.
POST	/approval-cases/{id}/reject	Reject with required reason.
GET	/exceptions	Action queue.
POST	/policies/{id}/versions	Create draft policy version.
POST	/policy-versions/{id}/publish	Publish effective-dated immutable version.
POST	/pay-periods/{id}/close	Validate and close period.
POST	/pay-periods/{id}/reopen	Privileged reopen with reason.
POST	/pay-periods/{id}/exports	Create/retrieve deterministic export artifact.
GET	/audit-events	Permission-controlled audit query.

10.3 Domain Events

Event	Emitted when	Typical consumers
TimeEventRecorded	New raw time event committed	Attendance recalculation, audit projection.
AttendanceRecalculated	Derived record changes materially	Exception workflow, reporting.
ExceptionOpened	Actionable rule finding created	Notification, manager queue.
ApprovalCompleted	Manager/HR decision finalized	Attendance finalization, notification.
PayPeriodClosed	All close validations pass	Payroll export readiness, reporting.
PayrollExportGenerated	Export artifact produced	Audit, external delivery adapter.

10.4 Integration Principles

- Inbound terminal/HRIS events require external source + stable external event ID for deduplication.
- Outbound integrations use retry with bounded exponential backoff and dead-letter/operator visibility.
- Webhook deliveries are signed and replay-protected.
- Do not place vendor-specific DTOs in domain modules; adapters map external contracts to internal commands.

11. Frontend Technical Architecture

11.1 Frontend Goals

- Fast, keyboard-accessible enterprise workflows for data-heavy attendance review.
- Type-safe boundary from API contract to UI models.
- Predictable separation of server state, URL state, form state and local UI state.
- Feature isolation so employee, scheduling, attendance and payroll areas can evolve independently.
- Observable and testable user flows with stable loading/error/empty states.

11.2 Recommended Stack

Concern	Decision	Best-practice intent
UI runtime	React 19.2.x	Modern component model; avoid class components for new work.
Language	TypeScript 6 with strict mode	No implicit any; type-safe contracts and discriminated states.
Build	Vite 8	Fast development/build tooling for SPA.
Routing	React Router v7 current stable line	Route-centric navigation, loaders only where they improve UX.
Design system	Material UI 9 + tenant-neutral theme tokens	Accessible primitives and consistent dense enterprise UI.
Server state	TanStack Query	Caching, invalidation, retry, request

Concern	Decision	Best-practice intent
		deduplication, async state.
Forms	React Hook Form + schema validation	Performant forms; validation schema shared conceptually with API contract.
Schema validation	Zod or equivalent runtime schema	Validate untrusted API/form boundaries.
Unit/component tests	Vitest + React Testing Library	Behavior-focused component tests.
E2E	Playwright	Critical cross-browser business flows.
API generation	OpenAPI-generated client/types where practical	Reduce drift between backend contract and frontend models.

11.3 Frontend Project Structure

Area	Contents
app/	App composition, providers, router, theme, error boundary, query client.
features/attendance/	Screens/components/hooks/types for attendance review and detail.
features/scheduling/	Schedule calendar/list, shift editor, assignment flows.
features/people/	Employee directory/profile and employment assignment.
features/payroll/	Pay period readiness, close/reopen, export history.
features/policies/	Policy draft/publish screens and typed rule editors.
shared/api/	Generated API client, transport interceptors, problem-error mapping.
shared/ui/	Design-system wrappers and reusable accessible primitives.
shared/lib/	Pure formatting, date/time, permission helpers; no feature business rules.
shared/test/	Factories, mock server handlers, test utilities.

11.4 State Ownership Rules

State type	Owner	Examples
Server state	TanStack Query	Employee list, attendance records, dashboard metrics.
URL state	Router/search params	Filters, sort, selected date range,

State type	Owner	Examples
		page/cursor when shareable.
Form state	React Hook Form	Policy editor, correction request, shift edit.
Local UI state	Component state	Dialog open/closed, current tab when not shareable.
Cross-cutting session	Small app context/store	Authenticated user summary, theme preference; avoid caching business entities globally.

11.5 Frontend Data Flow

STEP Route renders feature shell.

STEP Feature hook builds stable query key from tenant-safe identity and URL filters.

STEP TanStack Query calls typed API client.

STEP API errors map to normalized problem shape; global auth/network handling remains separate from business validation.

STEP Mutation invalidates or updates only related query keys.

STEP UI always has explicit loading, error, empty, stale and success states.

11.6 Date/Time Frontend Rules

- Frontend must never assume browser time zone equals employee/business time zone.
- All attendance detail views display the business date and zone label explicitly.
- Formatting utilities accept a ZoneId argument; do not scatter new Date() formatting across components.
- Ambiguous DST local times are displayed with zone/offset detail when relevant to troubleshooting.

12. UX / UI Requirements

12.1 Information Architecture

Navigation	Primary screens
Dashboard	Today overview, exceptions, payroll readiness.
People	Employee directory, profile, assignment.
Schedule	Calendar/list, shift template, assignment.
Attendance	Daily list, record detail, corrections, exceptions.
Approvals	Manager queue and decision detail.
Leave	Approved leave list / import (MVP-light).
Payroll	Pay periods, readiness, close/reopen, export history.

Navigation	Primary screens
Policies	Attendance policies, versions, holiday calendars.
Administration	Tenant, roles, locations, integrations, audit viewer.

12.2 Key Screen Requirements

Screen	Must show	Primary action
Employee dashboard	Current/next shift, today clock state, recent attendance exceptions	Clock in/out; submit correction.
Manager dashboard	Team status and action counts	Open exception/approval queue.
Attendance list	Employee, schedule, actual, worked, overtime, status, exception badges	Filter, inspect, bulk navigation.
Attendance detail	Timeline of raw events, calculated intervals, policy findings, decisions, audit	Approve/reject/correct where permitted.
Schedule planner	Employee/date grid or list, templates, overlap warnings	Assign/edit shift.
Policy editor	Effective dates and typed rule sections with preview examples	Save draft; validate; publish.
Payroll readiness	Finalized %, blockers, totals, changed-after-approval indicators	Close period.
Audit viewer	Who/what/when/correlation and safe diff summary	Filter/export audit evidence if authorized.

12.3 Accessibility and Interaction Standards

- Target WCAG 2.2 AA for core workflows.
- All actions keyboard reachable with visible focus.
- Do not encode status only by color; use text/icon labels.
- Tables have proper headers, accessible names and logical focus behavior.
- Dialogs trap focus correctly and restore focus on close.
- Validation errors are associated with fields and summarized for long forms.
- Loading indicators do not cause disruptive layout shift; long-running actions expose progress/state.

12.4 Responsive Behavior

- Desktop is primary for HR/payroll/manager workflows.
- Employee self-service supports mobile-width responsive layouts.
- Large data tables degrade to stacked summaries or horizontal scroll with sticky key columns; no hidden critical status.

13. Security and Compliance Controls

13.1 Authentication / Authorization

- OIDC/OAuth 2.0 integration with enterprise identity provider; backend acts as protected resource server.
- JWT or opaque bearer token validation follows Spring Security supported resource-server model.
- Audience, issuer, expiry and signing keys are validated; clock skew is bounded.
- Authorization is applied at use-case boundary and query scope, not only at controllers.
- Administrative permissions and org scopes are modeled explicitly.

13.2 Tenant Isolation

- Authenticated tenant claim/context is authoritative.
- Every tenant-owned repository operation requires tenant context; automated architecture/tests verify the convention.
- Cache keys include tenant identifier.
- Object storage/export paths include unguessable tenant-scoped namespaces.
- Cross-tenant attempts produce generic inaccessible/not-found semantics without leaking existence.
- Optionally enable PostgreSQL Row Level Security as defense-in-depth after evaluating operational complexity.

13.3 Application Security Baseline

- Use OWASP ASVS 5.x as verification checklist appropriate to application risk.
- Validate all untrusted input; use parameterized persistence APIs.
- CORS restricted to configured frontend origins; CSRF posture matched to token/cookie architecture.
- Secrets come from secret manager/environment injection, never repository files.
- Dependency scanning, SAST, container scanning and secret scanning run in CI.
- Security-relevant audit: login/identity mapping changes, role changes, policy publish, payroll close/reopen, export, support access.

13.4 Data Protection

- TLS in transit; managed database encryption at rest.
- Minimize PII in logs, traces, metrics and exported demo data.
- Demo repository ships synthetic data only.
- Retention policy distinguishes immutable audit/business obligations from disposable operational telemetry.
- Sensitive exports require authorization and are not exposed through permanent public URLs.

14. Non-Functional Requirements

ID	Category	Target / Requirement
NFR-01	Availability	Portfolio production target 99.9% monthly for API, excluding planned maintenance.
NFR-02	Latency	p95 read API < 400 ms and p95 command API < 700 ms under documented demo load, excluding external IdP.
NFR-03	Clock ingestion	Sustain burst ingestion without duplicate logical events; initial benchmark target 100 events/s on modest environment.
NFR-04	Calculation	Single employee/day recalculation should

ID	Category	Target / Requirement
		typically complete < 200 ms in-process under normal policy complexity.
NFR-05	Scalability	Stateless app instances scale horizontally; DB is primary consistency boundary.
NFR-06	Consistency	Clock append and authoritative attendance state are strongly consistent within designed transaction; notifications/integrations may be eventually consistent.
NFR-07	Recovery	Automated backups; target RPO <= 15 minutes and RTO <= 60 minutes for portfolio production environment.
NFR-08	Auditability	Every privileged or attendance-mutating action is attributable to subject, time, reason/correlation as applicable.
NFR-09	Accessibility	Core web workflows conform to WCAG 2.2 AA target.
NFR-10	Browser support	Latest stable Chrome, Edge, Firefox, Safari per enterprise support matrix.
NFR-11	Maintainability	Module dependency verification, architecture tests, static analysis and documented ADRs are part of CI.
NFR-12	Localization	English baseline; UI text externalized for future localization; dates/numbers formatted by locale while business zone remains explicit.

14.1 Capacity Model for Demonstration

- Seed demo: 3 tenants, 10,000 employees total, 90 days of schedules and time events.
- Benchmark query: tenant daily attendance list over 5,000 employees.
- Benchmark ingestion: retry-heavy batch with 5-10% duplicate submissions.
- Benchmark calculation: batch recalc for a pay period after policy/schedule correction.

15. Testing and Quality Strategy

15.1 Test Pyramid / Portfolio Evidence

Test layer	Focus	Examples
Pure unit	Domain rules and value objects	Late grace, overtime buckets, event pairing, business-date resolution.
Module/application	Use cases and module boundaries	Clock event -> recalculation; approval; policy publish.
Persistence	Real PostgreSQL behavior via Testcontainers	Unique idempotency, optimistic locking,

Test layer	Focus	Examples
		indexes/query behavior.
API contract	HTTP semantics and security	Problem responses, authz, validation, idempotency.
Frontend component	User-visible behavior	Filter state, validation, permission-driven actions.
E2E	Critical business journeys	Clock -> exception -> approve -> period close -> export.
Architecture	Dependency rules	No cyclic modules; domain does not depend on adapters.
Performance	Hot paths and concurrency	Clock burst, daily attendance list, concurrent approval.

15.2 Mandatory Edge-Case Test Catalogue

- Night shift crossing midnight.
- Spring-forward DST shift.
- Fall-back DST repeated local hour.
- Duplicate idempotency key same payload.
- Duplicate idempotency key different payload -> conflict.
- Out-of-order clock event.
- Missing clock-out and later correction.
- Overlapping schedules.
- Holiday plus overtime interaction.
- Approved leave covering full or partial shift.
- Policy effective-date boundary.
- Concurrent approve vs reject.
- Recalculation after schedule correction.
- Cross-tenant identifier probing.
- Closed period mutation blocked.
- Reopen then new export version.

15.3 Quality Gates

- Backend compile/test on Java 25.
- No failing architecture/module verification tests.
- Minimum agreed coverage is secondary to mutation/behavior evidence; critical domain rules require direct tests.
- Frontend TypeScript strict check, lint and unit/component tests.
- OpenAPI contract diff reviewed for breaking changes.
- Database migration validated from previous release snapshot.
- Container/image vulnerability thresholds enforced.

16. Observability and Operations

16.1 Telemetry

Signal	Required content
Structured logs	timestamp, level, service, environment, correlation_id, trace_id, tenant pseudonymous key, operation, safe entity IDs; never raw sensitive payloads.
Metrics	request rate/error/latency, DB pool, JVM, clock ingestion, recalculation duration, open exceptions, export failures, notification retry.
Traces	HTTP -> application use case -> DB/external integration spans; correlation carried across async work.
Audit	Business/security evidence separate from debug logs and protected by authorization/retention policy.

16.2 Operational Dashboards

- API RED metrics: rate, errors, duration.
- Clock ingestion rate, duplicate rate, rejected event rate.
- Attendance recalculation latency and failure queue.
- DB connections, slow query count, lock wait, replication lag if used.
- Payroll close/export success and duration.
- Notification/webhook retries and dead letters.

16.3 Alerting

- Sustained API 5xx above threshold.
- Clock ingestion failures or backlog.
- Database saturation / disk pressure.
- Repeated payroll export failure.
- Authentication/authorization error spike suggesting IdP or configuration incident.

17. DevSecOps and Deployment

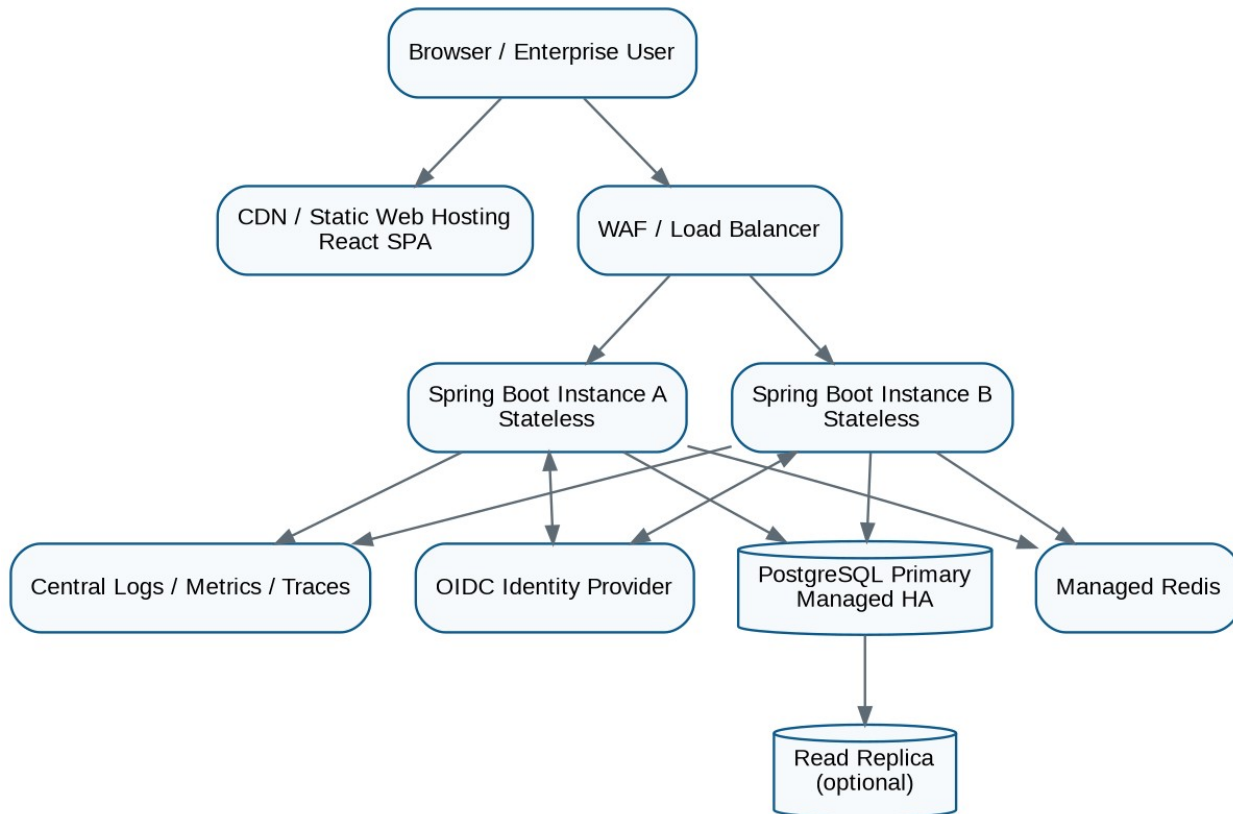


Figure 4. Production-like deployment model for the portfolio project.

17.1 Environments

Environment	Purpose	Data
Local	Developer workflow with Docker Compose dependencies	Synthetic local seed.
CI	Automated tests, migrations, security/quality gates	Ephemeral Testcontainers/synthetic.
Staging	Production-like end-to-end validation and demo rehearsal	Synthetic enterprise-sized dataset.
Production demo	Public/private portfolio demonstration	Synthetic only; no real employer/client data.

17.2 CI Pipeline

- STEP** Checkout and dependency verification.
- STEP** Backend formatting/static analysis, compile and unit tests on Java 25.
- STEP** Architecture/module verification tests.
- STEP** Testcontainers integration suite and migration verification.
- STEP** Frontend install with lockfile, lint, strict type check and tests.

STEP Build frontend and backend artifacts.

STEP SAST, dependency, secret and container scans.

STEP Generate OpenAPI/technical docs and compare contract changes.

STEP Build signed OCI image; publish only from protected branch/tag.

STEP Deploy staging; run smoke/E2E; promote intentionally.

17.3 Database Migration

- Flyway migrations are forward-only in normal delivery; rollback means application strategy plus corrective migration, not destructive ad-hoc SQL.
- Migrations are reviewed for lock/size impact.
- Large backfills are separated from schema-change transaction when necessary.
- App supports rolling deployment compatibility window where feasible.

17.4 Configuration

- 12-factor style environment configuration for deploy-specific values.
- Secrets stored in secret manager.
- Feature flags for incomplete integrations or staged rollout, not for permanent architecture forks.
- Tenant business rules belong in validated domain configuration, not environment variables.

18. Implementation Roadmap

Phase	Scope	Portfolio milestone
0 - Foundation	Repo, ADRs, CI, Java 25/Spring Boot, React/TS, Postgres, auth skeleton, module verification	Green build and architecture documentation.
1 - People & Schedule	Tenant, people, org, shift templates, schedules, basic UI	User can configure employees and planned work.
2 - Time Capture	Idempotent raw events, employee clock UI, timeline	Retries cannot duplicate logical punches.
3 - Attendance Engine	Calculation, policy strategies, cross-midnight/DST, exceptions	Core domain test suite is the centerpiece.
4 - Approval	Correction request, manager queue, optimistic locking, audit	Full exception lifecycle demo.
5 - Payroll	Pay period validation, close/reopen, deterministic CSV export	End-to-end business value complete.
6 - Production Quality	Observability, load tests, security gates, synthetic scale data, polished UX	Repository is interview-ready.
7 - Optional Advanced	Kafka/outbox, external HRIS/terminal adapters, read replica/report projection	Only after core is demonstrably strong.

18.1 Recommended Repository Deliverables

- README with product story, demo credentials, architecture and business flow.

- /docs/adr for architectural decisions: modular monolith, tenancy, time representation, idempotency, policy versioning, auth, payroll close.
- OpenAPI spec and generated API documentation.
- Database ERD and module diagram.
- Seed dataset and scripted demo scenario.
- Load-test scripts and benchmark report.
- Threat model / security checklist.
- Test catalogue mapping to domain edge cases.

19. Demo and Interview Narrative

19.1 Ten-Minute Demo Script

- STEP** Open tenant dashboard showing offices in Jakarta and Berlin.
- STEP** Show employee schedule with explicit IANA time zone and a cross-midnight shift.
- STEP** Clock in through employee view; repeat same request to demonstrate idempotent result.
- STEP** Open raw event timeline and derived attendance calculation.
- STEP** Trigger late/overtime condition and show rule-generated exceptions.
- STEP** Switch to manager role; approve overtime and demonstrate audit trail.
- STEP** Attempt a stale concurrent decision or explain optimistic locking conflict behavior.
- STEP** Open payroll readiness, close period, generate CSV export and show export version/checksum.
- STEP** Show tests for DST and cross-midnight cases plus module dependency verification.
- STEP** Finish with observability dashboard and architecture diagram.

19.2 Senior-Level Talking Points

Topic	Interview message
Architecture	I chose a modular monolith because distribution is a deployment concern, while domain modularity is needed immediately.
Time correctness	I model event time as Instant plus ZoneId/business date; DST and cross-midnight behavior is tested explicitly.
Reliability	Clock ingestion is idempotent because network/device retries are expected, not exceptional.
Data integrity	Raw events are immutable; corrections and recalculation preserve traceability.
Concurrency	Approvals use optimistic concurrency so a stale manager cannot silently overwrite another decision.
Extensibility	Attendance policy behavior is a composition of small strategies, applying OCP/DIP rather than giant conditionals.
Security	Tenant isolation is part of every layer and verified with negative tests.
Operations	The project includes metrics, logs, traces, migrations, CI gates

Topic	Interview message
	and recovery assumptions because production engineering is more than source code.

20. Acceptance Criteria

AC	Acceptance condition
AC-01	A synthetic tenant can configure at least two business locations with different IANA time zones.
AC-02	An employee can be scheduled and clock events can be recorded without duplicate logical events under retry.
AC-03	Attendance is correctly derived for normal, late, early-leave, overtime, absent, missing-punch and unscheduled-work scenarios.
AC-04	Cross-midnight and DST scenarios have deterministic automated tests.
AC-05	Manager can approve/reject an exception; stale concurrent update is rejected safely.
AC-06	Raw time events cannot be edited through ordinary business flows; correction history remains traceable.
AC-07	Tenant A user cannot read or mutate Tenant B entities, including by guessed IDs.
AC-08	Published policy versions are effective-dated and cannot be silently mutated historically.
AC-09	Payroll period close blocks unresolved required items and prevents normal edits after close.
AC-10	Export is reproducible/versioned and records who generated it.
AC-11	Core UI workflows are keyboard usable and meet targeted WCAG 2.2 AA checks.
AC-12	CI proves backend tests, frontend tests, architecture checks, migrations and security scans.
AC-13	Staging telemetry exposes API latency/error rate, clock ingestion, recalculation and payroll export health.
AC-14	Repository documentation lets an external senior engineer understand core trade-offs without reading every source file.

21. Risks and Trade-offs

Risk / Trade-off	Impact	Mitigation
Policy engine becomes too generic	Complex configuration and weak domain semantics	Start with typed rule strategies and explicit parameters; avoid building a generic DSL prematurely.
Shared-schema multi-tenancy bug	Potential severe data exposure	Tenant context at auth/use-case/repository/cache layers; negative integration tests; optional RLS defense-in-depth.
Time-zone edge cases	Incorrect hours/payroll	Instant + ZoneId model, central time service, deterministic DST/cross-midnight tests.
Mutable historical configuration	Non-reproducible payroll/audit	Effective-dated immutable published policy versions and schedule audit/versioning.
Microservice overengineering	Slower delivery and operational noise	Modular monolith until measurable extraction drivers exist.
Frontend global-state sprawl	Stale data and hard-to-test UI	TanStack Query for server state; URL/form/local state owned separately.
Demo scope explosion	Incomplete project looks weaker than smaller finished system	Protect P0 scope; advanced integrations and Kafka are optional.
Using newest framework versions	Migration/library compatibility risk	Pin tested versions with lockfiles/BOM; document upgrade policy and prefer supported stable releases.

22. Reference Technology Baseline

The following official documentation was checked when establishing the August 2026 baseline. Versions should remain pinned in the implementation repository and upgraded through reviewed dependency changes rather than floating tags.

Source	URL	Use in this PRD
Spring Boot System Requirements	https://docs.spring.io/spring-boot/system-requirements.html	Spring Boot 4.1.0 requires Java 17+ and is compatible through Java 26.
Oracle Java Downloads / Java SE Support Roadmap	https://www.oracle.com/java/technologies/downloads/	JDK 25 is the latest LTS line; Java 26 is a feature release.
Spring Modulith	https://spring.io/projects/spring-modulith/	Domain-driven application modules for well-structured Spring Boot applications; 2.1 GA available in 2026.
Spring Security OAuth 2.0 Resource Server	https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/	Spring-supported bearer token protection for APIs.
PostgreSQL 18 Documentation	https://www.postgresql.org/docs/18/	PostgreSQL 18 is the current stable major documentation line in August 2026.
React Versions	https://react.dev/versions	React 19.2.x is the current major/minor line;

Source	URL	Use in this PRD
		19.2.7 listed June 2026.
Vite 8	https://vite.dev/blog/announcing-vite8	Vite 8 released March 2026.
Material UI Versions	https://mui.com/material-ui/getting-started/versions/	Material UI v9 is the stable production-recommended major released April 2026.
TanStack Query React	https://tanstack.com/query/latest/docs/framework/react	Current React server-state/query documentation.
TypeScript 6.0 Release Notes	https://www.typescriptlang.org/docs/handbook/release-notes/typescript-6-0.html	TypeScript 6 documentation current in 2026.
WCAG 2.2	https://www.w3.org/TR/WCAG22/	W3C Recommendation used as accessibility target.
OWASP ASVS	https://owasp.org/www-project-application-security-verification-standard/	Application security verification baseline; use current 5.x checklist at implementation time.

22.1 Final Positioning Statement

What this project should communicate The strongest signal is not that the candidate can create forms and endpoints. It is that the candidate can reason about a global workforce domain, protect invariants, model time correctly, design for retries/concurrency/auditability, keep a codebase modular under SOLID, build an accessible enterprise frontend, and operate the system as if real customers depended on it.

Appendix A - Status and Exception Vocabulary

Type	Meaning
NORMAL	Attendance meets configured expectations.
LATE	Arrival exceeds configured grace/tolerance.
EARLY_LEAVE	Departure occurs before allowed threshold.
ABSENT	Scheduled work lacks qualifying time or leave.
MISSING_CLOCK_IN	Expected first event absent/inconsistent.
MISSING_CLOCK_OUT	Expected closing event absent.
OVERTIME	Worked minutes exceed configured threshold and may require approval.
UNSCHEDULED_WORK	Time events exist without qualifying schedule.
HOLIDAY_WORK	Worked time falls in applicable holiday calendar.
REST_DAY_WORK	Worked time falls on configured rest day.
BREAK_VIOLATION	Break duration/timing violates policy.
OVERLAPPING_SHIFT	Schedule entries conflict and require correction.

Appendix B - Example Synthetic Demo Scenario

- Tenant: Global Industrial Demo. Locations: Jakarta (Asia/Jakarta), Berlin (Europe/Berlin), London (Europe/London).
- Employee A: Jakarta day shift 08:00-17:00, 10-minute grace, overtime after 8 worked hours.
- Employee B: Jakarta night shift 22:00-06:00 crossing business date boundary.
- Employee C: Berlin schedule on a DST transition test date using synthetic historic/future-safe fixture.
- Manager queue contains late arrival, overtime approval and missing clock-out cases.
- Payroll period closes only after those required exceptions are resolved.

Appendix C - Definition of Done for Each Feature

- Business rule and acceptance criteria documented.
- Backend use-case/domain tests complete.
- Authorization and tenant-scope tests complete.
- API contract documented and error behavior defined.
- Frontend loading/error/empty/success states implemented and tested.
- Accessibility checks included for interactive workflow.
- Telemetry and audit behavior defined.
- Database migration and rollback/corrective strategy reviewed.
- README/demo data updated when feature is user-visible.